Failure (search fails mid-run) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `search_failure_practice.py`: a checkpointer's whole promise is that a crash doesn't erase earlier progress — this exercise proves it, by making search fail on purpose. **If not:** the Build Task's first real outage would also be your first time finding out what's left after a crash.

Basic Version

Approach 1 — the direct way

```
# search_failure_practice.py
from typing import TypedDict

from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import END, START, StateGraph


class FailureState(TypedDict):
    task: str
    plan: str
    sources: list


def plan_node(state):
    return {"plan": "search the documents for: " + state["task"]}


def search_node(state):
    # temporary, for this test
    raise RuntimeError("simulated vector store outage")


builder = StateGraph(FailureState)
builder.add_node("plan_node", plan_node)
builder.add_node("search_node", search_node)
builder.add_edge(START, "plan_node")
builder.add_edge("plan_node", "search_node")
builder.add_edge("search_node", END)
```

```
graph = builder.compile(checkpointer=MemorySaver())

config = {"configurable": {"thread_id": "failure-test-1"}}
try:
    graph.invoke({"task": "How hot should the water be?"}, config)
except RuntimeError as e:
    print("Caught expected failure:", e)

snapshot = graph.get_state(config)
print("Surviving state:", snapshot.values)
print("Was about to run:", snapshot.next)
```

Expected output:

```
Caught expected failure: simulated vector store outage
Surviving state: {'task': 'How hot should the water be?', 'plan': 'search
the documents for: How hot should the water be?'}
Was about to run: ('search_node',)
```

(Wrapped to fit the page.) `plan_node` finished before the crash, so its `plan` is saved; `get_state(config).next` shows the run stopped right before `search_node` (Doc09). This prints the state for you to check by eye — it doesn't test anything yet.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — real assertions, and a test that fails if the crash stops happening

```
# search_failure_practice.py
from typing import TypedDict

from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import END, START, StateGraph


class FailureState(TypedDict):
    task: str
    plan: str
    sources: list[dict]


def plan_node(state: FailureState) -> dict:
    return {"plan": f"search the documents for: {state['task']}"}


def broken_search_node(state: FailureState) -> dict:
    # temporary, for this test
    raise RuntimeError("simulated vector store outage")


def build_graph(search_node):
    builder = StateGraph(FailureState)
    builder.add_node("plan_node", plan_node)
    builder.add_node("search_node", search_node)
    builder.add_edge(START, "plan_node")
    builder.add_edge("plan_node", "search_node")
    builder.add_edge("search_node", END)
    return builder.compile(checkpointer=MemorySaver())


def test_state_survives_search_failure() -> None:
    graph = build_graph(broken_search_node)
    config = {"configurable": {"thread_id": "failure-test-1"}}
    try:
        graph.invoke({"task": "How hot should the water be?"}, config)
```

```
        raise AssertionError("expected the graph to raise, but it didn't")
    except RuntimeError as e:
        print(f"Caught expected failure: {e}")

    snapshot = graph.get_state(config)
    assert snapshot.values["plan"].startswith("search the documents")
    assert snapshot.next == ("search_node",)
    print("Confirmed: plan_node's work survived, and the graph knows")
    print("it stopped before search_node.")


if __name__ == "__main__":
    test_state_survives_search_failure()
```

Expected output:

```
Caught expected failure: simulated vector store outage
Confirmed: plan_node's work survived, and the graph knows
it stopped before search_node.
```

Difference from Basic: real `assert` checks instead of prints, wrapped in a test function you can re-run. The `raise AssertionError(...)` after `invoke()` matters: without it, a change that stopped the node from failing would make this test pass without testing anything. `build_graph(search_node)` takes the search node as a parameter, so the next approaches can plug in a real one.

Approach 2 — a retry with backoff, for temporary errors only

Story: a vector store down for 200 ms is nothing like a malformed query that fails the same way every time. **If not:** one brief network blip would fail a whole user-facing run that a simple retry would have saved.

Add below Approach 1's code:

```
# search_failure_practice.py
import time

from kb import collection
from logging_setup import get_logger
from retriever import retrieve

TRANSIENT_ERRORS = (TimeoutError, ConnectionError)
MAX_ATTEMPTS = 3
```

```
def real_search(query: str) -> list[dict]:
    return retrieve(collection, query)


def search_with_retry(search, query: str) -> list[dict]:
    logger = get_logger(__name__)
    for attempt in range(MAX_ATTEMPTS):
        try:
            return search(query)
        except TRANSIENT_ERRORS as exc:
            if attempt == MAX_ATTEMPTS - 1:
                raise
            wait_seconds = 2 ** attempt
            logger.warning(
                f"search attempt {attempt + 1} failed ({exc}); "
                f"retrying in {wait_seconds}s"
            )
            time.sleep(wait_seconds)
    raise RuntimeError("search_with_retry left its loop unexpectedly")


def make_search_node(search):
    def search_node(state: FailureState) -> dict:
        return {"sources": search_with_retry(search, state["task"])}
    return search_node


graph = build_graph(make_search_node(real_search))
config = {"configurable": {"thread_id": "retry-1"}}
result = graph.invoke({"task": "How hot should the water be?"}, config)
print(f"found {len(result['sources'])} sources")
```

Expected output:

```
found 3 sources
```

`TimeoutError` and `ConnectionError` get up to `MAX_ATTEMPTS` tries, waiting 1 s, then 2 s between them (Doc02's backoff). Anything else — a bug, a bad query — isn't in `TRANSIENT_ERRORS`, so it's raised on the first try; retrying a bug only wastes time. `make_search_node(search)` is Doc07's factory idea: it builds a node around whichever search function you pass in.

Approach 3 — proving every outcome with a fake, flaky search

Story: retry logic with no test can quietly start retrying permanent errors, or stop retrying temporary ones. **If not:** you'd find out only during a real outage.

`FlakySearch` is Doc07's `FlakyToolState` idea: a small class with a counter that fails a set number of times, then really searches. Add below Approach 2's code:

```
# search_failure_practice.py
class FlakySearch:
    """Fails the first `fail_times` calls with `error`, then searches."""

    def __init__(self, fail_times: int, error: Exception) -> None:
        self.fail_times = fail_times
        self.error = error
        self.call_count = 0

    def search(self, query: str) -> list[dict]:
        self.call_count = self.call_count + 1
        if self.call_count <= self.fail_times:
            raise self.error
        return real_search(query)


def test_recovers_from_transient_failure() -> None:
    flaky = FlakySearch(2, TimeoutError("simulated timeout"))
    results = search_with_retry(flaky.search, "How hot should the water be?")
    assert len(results) > 0
    assert flaky.call_count == 3
    print("Recovered after 2 timeouts, on the 3rd try.")


def test_permanent_failure_does_not_retry() -> None:
    flaky = FlakySearch(1, ValueError("malformed query"))
    try:
        search_with_retry(flaky.search, "anything")
        raise AssertionError("expected a ValueError, but none was raised")
    except ValueError:
        assert flaky.call_count == 1
        print("A permanent error was raised at once, with no retry.")


def test_state_survives_when_retries_are_exhausted() -> None:
    flaky = FlakySearch(99, TimeoutError("still down"))
    graph = build_graph(make_search_node(flaky.search))
    config = {"configurable": {"thread_id": "exhausted-1"}}
    try:
        graph.invoke({"task": "How hot should the water be?"}, config)
        raise AssertionError("expected the graph to raise a TimeoutError")
```

```
    except TimeoutError:
        pass
    snapshot = graph.get_state(config)
    assert flaky.call_count == MAX_ATTEMPTS
    assert snapshot.next == ("search_node",)
    print("Retries used up, and plan_node's work is still saved.")


test_recovers_from_transient_failure()
test_permanent_failure_does_not_retry()
test_state_survives_when_retries_are_exhausted()
```

Expected output (with two retry warnings logged before the first line, and two more before the last):

```
Recovered after 2 timeouts, on the 3rd try.
A permanent error was raised at once, with no retry.
Retries used up, and plan_node's work is still saved.
```

`call_count` is what proves the behavior: 3 calls for the recovery, exactly 1 for the permanent error, and exactly `MAX_ATTEMPTS` before giving up.

Difference from Approach 1, and between Approaches 2/3: Approach 1 proves the checkpointer's guarantee once. Approach 2 adds the retry itself. Approach 3 proves all three outcomes — recovery, no wasted retries, and saved state even when every retry fails.

Which one should you actually write? Keep Approach 1's crash test — it's the simplest proof checkpointing works at all. Add Approach 2's retry once search is a real network service. Add Approach 3's tests as soon as anyone (including future you) might change the retry logic.